3 STACKS

3.1 INTRODUCTION

A stack is a linear data structure. It is very useful in many applications of computer science. It is a list in which all insertions and deletions are made at one end, called the top of the stack. Some of the analogies used to visualize this data structure are

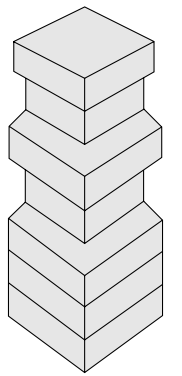


Fig. 3.1 *Stack of plates*

a) Stack of Trays (or) Plates Placed on a Table Here, one plate is placed on top of another, thus creating a stack of plates. Suppose, a person takes a plate off the top of the stack of plates. The plate most recently placed on the stack is the first one to be taken off. The bottom plate is the first one placed on the stack and the last one to be removed.

b) Another Familiar Example of a Stack is a Railway Station for Shunting Cars In this example, the last railway car placed on the stack is the first to leave.

c) Shipment in a Cargo For the shipment of goods, they have to be loaded into a cargo. During unloading, they are unloaded exactly in the opposite order as they are loaded, that is, the goods which are loaded last should be unloaded first.

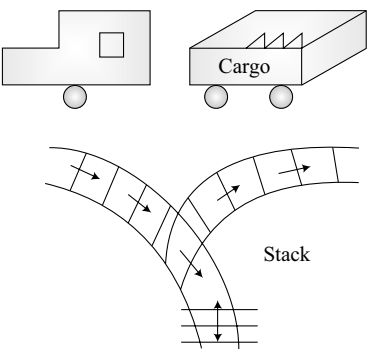


Fig. 3.2 *A railway shunting system—represenation of a stack*

3.2 DEFINITION

A stack is an ordered collection of homogeneous data elements where the insertion and deletion operations occur at one end only, called the top of the stack. Fig. 3.3 shows the schematic diagram of a stack.

The alternative representation of a stack is given in Figure. 3.4.

Other names for a stack are *pushdown list, and LIFO* (or) Last In First Out list. The stack allows access to only one item, i.e., the last item inserted. In the schematic diagram of a stack, after inserting Item 4 into the stack, it acts as the topmost element. It will be removed first. The very first item that was inserted into the stack is Item 1. It will be removed as the last item.

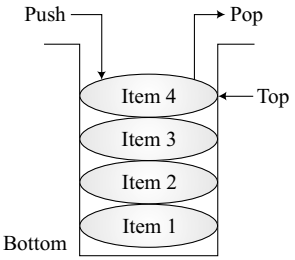


Fig. 3.3 *Schematic diagram of a stack*

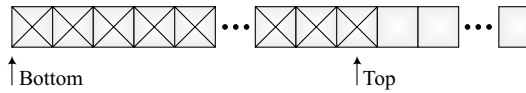


Fig. 3.4 *Alternative representation of a stack*

3.3 PRIMITIVE OPERATIONS

The primitive operations that can be performed on a stack are given below:

1. Inserting an element into the stack (PUSH operation)
2. Removing an element from the stack (POP operation)
3. Determining the top item of a stack without removing it from the stack (PEEP operation)

The syntax used for the PUSH operation is

```
PUSH (stack, item)
```

where **stack** is the name of the stack into which the item specified as the second argument is placed at the top position of the stack.

The syntax used for the POP operation is

```
POP(stack)
```

where **stack** is the name of the stack from which the item has to be removed, i.e., the element at the topmost position of the stack is removed. Consider a stack structure as given in Figure 3.5(a). After inserting (PUSH) an element (viz, 55) into this stack, the contents of the stack after the PUSH operation is given in Figure 3.5(b).

Suppose if the POP operation is specified as POP(st). Then the content of the stack can be visualized as in Figure 3.5(c).

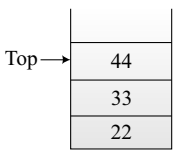


Fig. 3.5(a)

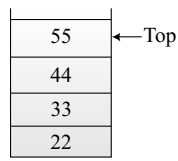


Fig. 3.5(b)

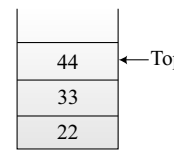


Fig. 3.5(c)

That is, the element pointed out by Top will be removed and then Top will be decremented by 1. So it points the item placed below the removed item. It is also possible to verify the item placed at the top of the stack without removing it. This operation is called PEEP. The syntax used for this operation is

```
PEEP(st)
```

This operation is the combination of POP and PUSH. It is equivalent to the following statements:

```
i = POP(st)
PUSH(st,i)
```

POP removes the item from the position pointed out by Top. It is in variable **i**. Then using the PUSH instruction we are inserting the same item **i** into the stack. Now the variable **i** has the value in the topmost position of the stack.

3.3.1 Other Operations

The other operations that can be performed on the stack are the following:

1) Checking the Emptiness of the Stack It is not possible to pop elements from the empty stack. So at the time of issuing the POP operation, the stack should be non-empty. The syntax used to check the empty condition of the stack is

```
Empty(stack)
```

It returns TRUE if the stack is empty and FALSE otherwise.

2) Checking the Full Condition of the Stack If the stack is full, it is not possible to push elements into it. So before issuing the PUSH operation, make sure that the stack is not full. The syntax used to check the full condition of the stack is

```
Full (stack)
```

It returns TRUE if the stack is Full and FALSE otherwise.

3) Before Using the Stack, the Size of the Stack and the Variable which Points the Top Element of the Stack have to be Initialized

3.4 AN ABSTRACT DATA TYPE (ADT)

An Abstract Data Type (ADT) is a keyword used in a programming language to specify the amount of memory needed to store data and the type of data that will be stored in that memory location. However, an ADT does not specify how many bytes to reserve for the data, because the number of bytes reserved for an ADT varies from one programming language to another.

ADT for Stack This is a Last-In First-Out structure. Items can only be inserted and removed from the top of the stack. The ADT for stack are the following:

1) Push() (or) Put (or) insert new item on top of stack
2) Pop() remove and return top item of stack
3) Top() Verifies the top item of stack

3.5 IMPLEMENTATION

A stack can be implemented in any one of the following two ways :

1. Using an array
2. Using a linked list

3.5.1 Array Implementation of Stacks

In the array implementation of a stack, a stack can grow either towards the high-indexed end of the array (or) towards the low-indexed end of the array. The array implementation of the stack which grows towards the low indexed end of the array is shown in Fig. 3.6.

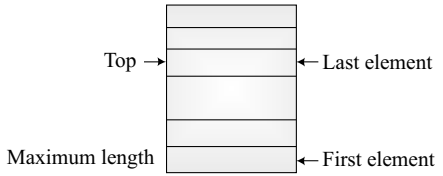


Fig. 3.6 *An array implementation of a stack*

The array implementation requires the following.

1) An array
2) A variable top to hold the topmost element of the stack

Here, the stack is of fixed size. This means that some maximum limit is specified for storing the elements into the stack. Once the maximum limit is reached, it is not possible to store the elements into it. In Figure 3.6, the low-indexed end of the array holds the last element that can be retrieved from the array, i.e., the very first element that was pushed into the stack occupies the last index position of the array. The last element that is pushed into the stack is pointed out by the variable top. It is the first to be removed from the stack. The array implementation has the following disadvantages.

1. Memory space is wasted (because of storing less number of items in the array than the maximum size of the array).
2. There is a limitation in storing the items into the stack.
3. Insertion and deletion operations in a stack using an array involves more data movements.

The advantage of this method is that the implementation of the stack using an array is easy. A sample program that illustrates the array implementation of a stack is given below:

Program /* Array implementation of a stack */

```
#include<stdio.h>
#include<conio.h>
   int s[20],item,i,n=10;
       extern top=-1;
main()
       {
       while(1)
       {
       clrscr();
           printf("Array implementation of a stack\n ");
           printf("1. Insertion \n");
           printf("2. Deletion \n");
           printf("3. Display \n");
           printf("4. Exit \n");
           printf("Enter your choice (1..4) ");
           scanf("%d",&i);
           switch(i)
```

```
            {
            case 1:
                printf("Insertion \n");
                printf("Enter Item ");
                scanf("%d",&item);
                insert(item);
                break;
            case 2:
                printf("Deletion\n");
                dele();
                break;
            case 3:
                display();
                break;
        case 4:
                exit(1);
                }
            }
        }
insert(item)
{
        if(top>=n)
            printf("Stack Full");
        top=top+1;
        s[top]=item;
        return;
}
dele()
{
        if(top<0)
        {
        printf("Stack empty");
        return;
        }
        item=s[top];
        printf("Deleted item is %d",item);
        getch();
        top=top-1;
        return;
}
```

/*Function used to display the content of the stack in the LIFO manner*/

```
display()
{
int i=0;
printf("Content of the stack is :");
while(i <= top)
{
printf("%d ",s[i]);
i++;
}
getch();
return;
}
```

Sample Input and Output Array implementation of a stack

1. Insertion
2. Deletion
3. Display
4. Exit

Enter your choice (1..4)1
Insertion
Enter Item 11
Array implementation of a stack

1. Insertion
2. Deletion
3. Display
4. Exit

Enter your choice (1..4)1
Insertion
Enter Item 12
Array implementation of a stack

1. Insertion
2. Deletion
3. Display
4. Exit

Enter your choice (1..4)3
Content of the stack is :11 12
Array implementation of a stack

1. Insertion
2. Deletion
3. Display
4. Exit

Enter your choice (1..4) 4

3.5.2 Linked List Implementation of a Stack

In the case of linked-list implementation of a stack, the first element inserted into the stack is pointed out by the second element, the second element by the third, and so on. In general the (**n**-1)th element is pointed out by the nth element. The pointer address corresponding to the **n**th element has to be maintained. The linked-list implementation of a stack is as follows.

The linked-list implementation of a stack requires the following:

1. Structure declaration containing some **n** fields.
2. Pointer to the first node (or) the last node.

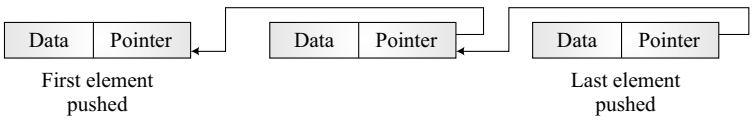


Fig. 3.7 *Linked-list implementation of a stack*

Using this, the linked list can be traversed. Here, the stack does not need to be of fixed size. There can be any number of elements or nodes in the stack. This is the notable advantage of the linked-list implementation of a stack. The second advantage of this method is that insertion and deletion operations do not involve more data movements (just the change of pointers). The third advantage of this method is that memory space is not wasted, because memory space is allocated only when the user wants to push an element into the stack.

The disadvantage of this method is that more space is used for each node (because pointer fields are involved). In this arrangement, initially memory is allocated for the first node that has to be pushed into the stack. Here, each node can have the following fields.

a) Data field

b) Pointer field

For the first node, the pointer field will have the value NULL because it cannot point to any node (that is, no previous node exists for the first node). When the user creates the next node, memory space for this node should be allocated, and the starting address of this node should be placed in the pointer field of the previously created node (that is, in the immediate predecessor node) When the user wants to remove or pop an element from the stack, the node pointed out by the pointer **Top** is removed and then **Top** should be modified such that it should have the address of the immediate predecessor node of this deleted node.

The disadvantage of this method is that, in each node some space is used by the pointer field whereas in the array implementation, no pointer field is used. Comparing the advantages and disadvantages of this method and the array implementation, this linked-list method of implementation has more advantages. So this linked-list method of implementation is an efficient one than the array implementation of a stack. The following program illustrates the implementation of a stack using a linked list.

Program /* Linked list implementation of a stack. */

```
#include <stdio.h>
#include <conio.h>
#include <alloc.h>
```

/* structure containing data part and link part */

```
struct node
{
   int d ;
   struct node *link ;
};
struct node *top = NULL;
void push(int) ;
int pop();
void display();
void main( )
{
   int item,ch;
   clrscr( );
   while(1)
   {
```

```
   printf("Linked list implementation of a stack\n");
   printf("1.Push......\n");
   printf("2.Pop.......\n");
   printf("3.Display...\n");
   printf("4.Exit......\n");
   printf("Enter your choice(1..4) ");
   scanf("%d",&ch);
   switch(ch)
{
   case 1:
       printf("Enter the item to be inserted into the stack : ");
       scanf("%d",&item);
       push(item);
       break;
   case 2:
       item = pop();
       printf ( "\nItem popped: %d",item) ;
       getch();
       break;
   case 3:
       display();
       getch();
       break;
   case 4:
       exit(1);
   }
   }
}
```

/* Function used to add a new node to the stack */

```
void push (int item)
{
   struct node *temp ;
   temp = ( struct node * ) malloc ( sizeof ( struct node ) ) ;
   if (temp == NULL)
       printf ( "\nStack is full" ) ;
   temp -> data = item ;
   temp -> link = top ;
   top = temp ;
}
```

/* Function pops an element from the stack */

```
int pop()
{
   struct node *temp ;
   int item ;
   if (top == NULL)
   {
       printf ( "\nStack is empty." ) ;
       return NULL ;
   }
   temp = top;
   item =temp -> data;
   top = top -> link;
   free (temp) ;
   return item ;
}
```

/* Function used to display the content of the stack */

```
void display ()
{
   struct node *top1;
   int item;
   top1 = top;
   if (top1 == NULL)
       return;
   while (top1 != NULL)
   {
       item = top1 -> d;
       printf("%d ",item);
       top1 = top1 -> link ;
   }
}
```

Sample Input and Output Linked list implementation of a stack

1. Push
2. Pop
3. Display
4. Exit

Enter your choice(1..4) 1

Enter the item to be inserted into the stack : 11

Linked list implementation of a stack

1. Push
2. Pop
3. Display
4. Exit

Enter your choice(1..4) 3

11

Linked list implementation of a stack

1. Push
2. Pop
3. Display
4. Exit

Enter your choice(1..4) 4

3.6 APPLICATIONS OF STACK

In this subsection, three applications of stacks are explained. They are the following:

1. Infix to postfix conversion
2. Evaluation of arithmetic expressions
3. Decimal to binary conversion

3.6.1 Infix to Postfix Conversion

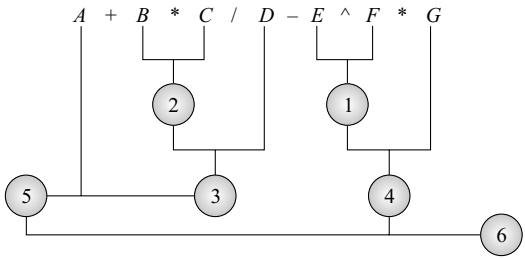
An arithmetic expression is the combination of variables, constants and arithmetic operators. The arithmetic operators used are

+	(addition)	–	(subtraction)
*	(multiplication)	/	(division)
^	(exponentiation)	%	(Remainder operator)

Variables and constants may be preceded by unary operators like –, +. To evaluate any arithmetic expression, the order of precedence and associativity of the operators should be known. This is as follows.

Operators	*Precedence*	*Associativity*
– (unary) + (unary)	4	-
^ (Exponentiation)	3	Right to left
*, /	2	Left to right
+, –	1	Left to right

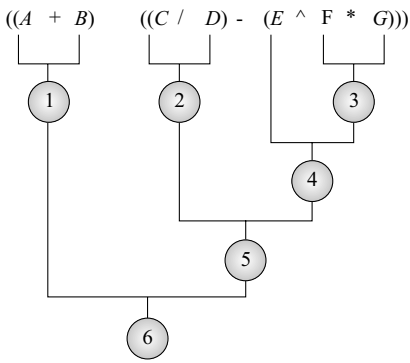
Example The arithmetic expression $A + B * C/D - E \wedge F * G$ is evaluated as follows:



To override the rules of precedence and associativity, the arithmetic expression may be parenthesized as follows.

$$((A + B) * ((C/D) - (E \wedge (F * G))))$$

With these parenthesization, the innermost parenthesis part will be evaluated first, then the next innermost, and so on.



The problem in evaluating the arithmetic expression of this form is that the expression must be scanned from left to right repeatedly. To overcome this problem, the following should be done.

a) Conversion of a given expression into a special notation
b) Evaluation of that new expression using a stack